intelligent-document-finetuning

Architecture guide: how the project is organized, why, and what changed

Prepared 30 October 2026

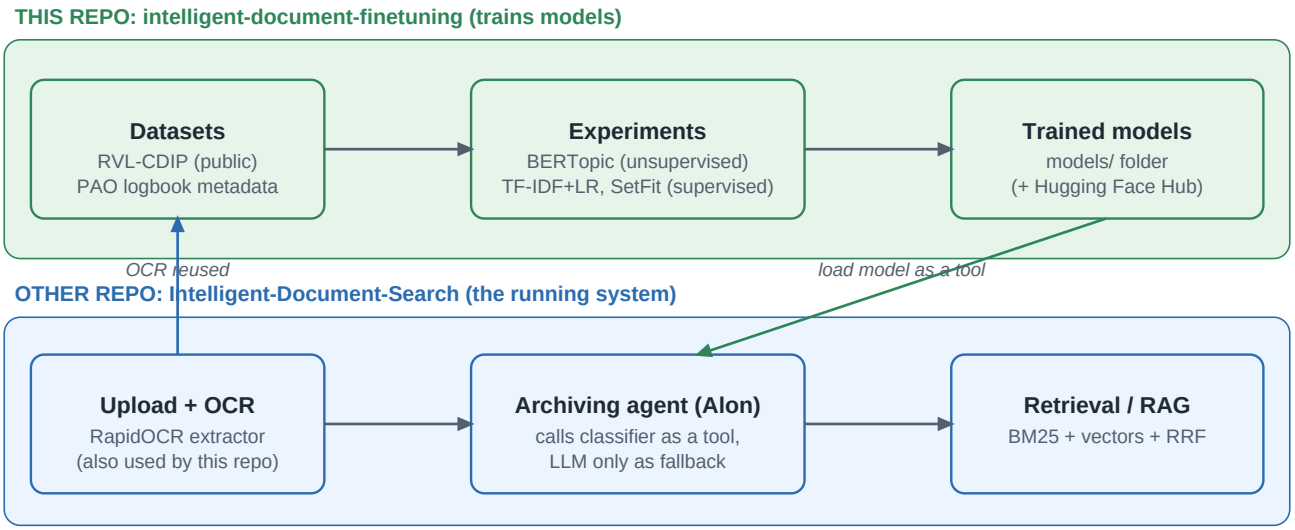
Summary

This repo is the **training lab** of your thesis. It does not run the archiving system. It prepares datasets, runs experiments, and produces trained models (BERTopic now; TF-IDF + LR and SetFit next) that the archiving agent in **Intelligent-Document-Search** will load and call as tools.

I reorganized the folders so that each kind of thing has exactly one home, grouped by task. **I did not change how any of your code works.** I only moved files, fixed the paths that the moves affected, fixed one typo bug, and added explanations (README, config values, notes). Your data, trained models, and the 37 GB RVL-CDIP archive were not touched. Nothing is committed yet.

1. The big picture

Two repos, two jobs. This one **learns**; the other one **serves**.



Why split them? Training needs heavy libraries, GPUs, big datasets and lots of trial and error. The production backend must stay small, stable and fast. Keeping them apart means an experiment can never break the running system. The only thing that crosses over is the finished model file.

2. The folder structure

Read this tree top to bottom. The comment on each line says what belongs there.

```
intelligent-document-finetuning/
|-- README.md               map of the repo + how to run things
|-- pyproject.toml          dependencies (managed by uv)
|
|-- configs/                SETTINGS - numbers you might cite in the thesis
|   |-- data/               rvl_cdip.yaml, pao_metadata.yaml
|   |-- topic_modeling/     bertopic_minilm.yaml
|   |-- classification/     tfidf_logreg.yaml, setfit.yaml
|   `-- eval/               classification.yaml, topic_discovery.yaml
|
|-- data/                   DATA - never committed to git
|   |-- raw/                original files, never edited
|   |   |-- rvl_cdip/       public benchmark (37 GB archive + 1,600 .tif)
|   |   `-- pao_metadata/   Book1.xlsx (private PAO logbook)
|   |-- interim/            half-way results (manifest, OCR log, embeddings)
|   `-- processed/          final model-ready tables (.parquet)
|
|-- notebooks/              EXPLORATION - numbered in the order you did them
|   |-- 01_pao_metadata_preparation.ipynb
|   `-- 02_pao_baseline_tfidf_logreg.ipynb
|
|-- scripts/                BUTTONS - small files you run from the terminal
|   |-- rvl_cdip/           01_download..., 02_prepare...corpus.py
|   |-- topic_modeling/     fit_bertopic.py
|   `-- debug/              one-off checks and early prototypes
|
|-- src/intelligent_document_finetuning/    TOOLBOX - reusable code
|   |-- ocr/                client.py, backend_runner.py
|   |-- data/               (next) dataset loading + cleaning
|   |-- topic_modeling/     (next) BERTopic building blocks
|   |-- classification/     (next) TF-IDF+LR and SetFit
|   `-- evaluation/         (next) shared CV + metrics
|
|-- models/                 RESULTS - trained models, one folder per run
|   `-- bertopic/v1_default ... v4_clean_vocab
|-- notes/                  FINDINGS - experiment logs for the thesis
|-- tests/                  automated tests (empty for now)
`-- docs/                   this guide
```

3. The five patterns behind this structure

These are standard in ML projects. Once you know them, any ML repo becomes easy to read.

Pattern 1: Separate code, data, settings and results

Code goes in **src/** and **scripts/**, data in **data/**, settings in **configs/**, results in **models/** and **notes/**. Mixing them is what makes projects confusing: you can't tell whether a file is something you wrote, something you downloaded, or something a script produced. Git also handles them differently: code is committed; data and models are not (too big, and the PAO data is private).

Pattern 2: Notebook = whiteboard, src = toolbox, scripts = buttons

Notebooks are for exploring: trying ideas, looking at data, seeing errors. That's exactly how you used them for the PAO metadata, which is right. Once an idea works, the reusable part moves into **src/** as a function (the toolbox). A **script** is then a short file that picks tools from the toolbox and runs them with specific settings (a button you press). The rule: scripts and notebooks import from src/; src/ never imports from scripts/. Then the agent backend can later import the same src/ code too.

Pattern 3: Data flows one way: raw -> interim -> processed -> models

raw/ is never edited, so you can always start over. Each step reads from the folder before it and writes to the next. Your RVL-CDIP pipeline already does this perfectly: tar.gz (raw) -> manifest + OCR log (interim) -> rvl_cdip_v1.parquet (processed) -> models/bertopic/<run>. The PAO data should follow the same path: save the cleaned, reviewed table to data/processed/pao_metadata_v1.parquet instead of keeping it only inside a notebook.

Pattern 4: Organize by task, not by file type

Folders are named after **what the code does** for your thesis: ocr, topic_modeling, classification, evaluation. The same names repeat in scripts/, src/ and configs/, so if you are working on classification you look in the 'classification' folder everywhere. Your old layout had generic folders (tools, utils, training, export, models/llm) that were empty and didn't say which task they belonged to.

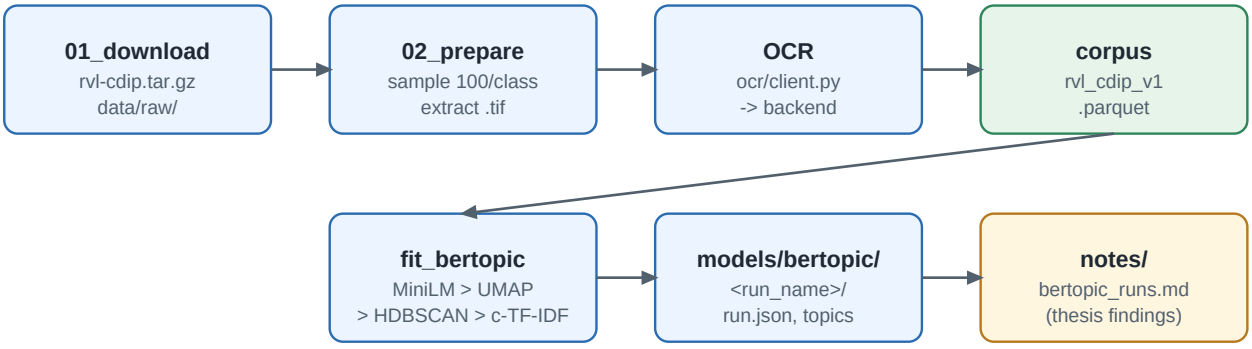
Pattern 5: Every run is reproducible and recorded

Fixed seeds (42), one output folder per run name (models/bertopic/v2_mcs10/), a run.json with the exact settings and scores, and a notes file explaining the result. You already do this for BERTopic, and it's the strongest part of the repo. A panel can ask 'how did you get this number?' and you can answer with a file. The classification work should get the same treatment (notes/classification_runs.md is started).

4. How data moves through the project

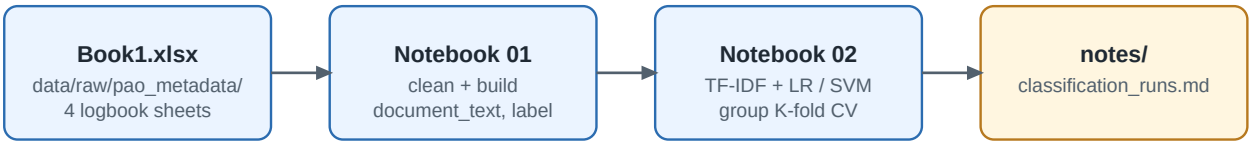
Track A: RVL-CDIP + BERTopic (done)

Public benchmark used to develop and test the unsupervised pipeline before using PAO data.



Track B: PAO metadata classification (in progress)

Your own office data. Currently notebook-based; the next step is to save the cleaned table to data/processed/ and move the model code into src/classification/.



How OCR works here (the one tricky part)

The OCR engine lives in your other repo, which has its own Python environment. So this repo doesn't import it directly. Instead, **ocr/client.py** starts **ocr/backend_runner.py** using the other repo's Python, sends it image paths, and reads back JSON lines. That's why backend_runner.py must stay a standalone file. Both files now sit together in src/.../ocr/, so the pair is easy to find.

Note: client.py and backend_runner.py contain your Windows path to Intelligent-Document-Search. If you move that repo or use another computer, update those two lines.

5. What I changed

Moved files (old location -> new location)

Old	New	Why
scripts/download_rvl_cdip.py	scripts/rvl_cdip/01_download_rvl_cdip.py	Grouped by dataset; numbered in run order
scripts/prepare_document_corpus.py	scripts/rvl_cdip/02_prepare_rvl_cdip_corpus.py	Same; name now says which corpus
scripts/fit_bertopic.py	scripts/topic_modeling/fit_bertopic.py	Grouped by task
scripts/test_extract_one.py	scripts/debug/test_ocr_one.py	It's a one-off OCR check
main.py (root)	scripts/debug/view_topic_info.py	Root main.py suggests an app entry point; it was a quick viewer
src/.../data/rvl_cdip_adapter.py	scripts/debug/sample_rvl_cdip_index.py	Runs code on import, so it's a script, not library code; superseded by 02_prepare
src/.../data/rvl_cdip_extractor.py	scripts/debug/extract_one_rvl_cdip_image.py	Same reason
src/ocr_client.py	src/.../ocr/client.py	Moved inside the package, next to its runner
src/.../tools/backend_ocr_runner.py	src/.../ocr/backend_runner.py	Grouped with the client
scripts/metadata_model/metadata_preparation.ipynb	notebooks/01_pao_metadata_preparation.ipynb	Notebooks get their own folder, numbered
scripts/inspect_classification_data.ipynb	notebooks/02_pao_baseline_tfidf_logreg.ipynb	Name now says what it contains
data/raw/classification/Book1.xlsx	data/raw/pao_metadata/Book1.xlsx	One home for the PAO Excel file
data/raw/metadata_model/Book1.xlsx	_to_delete/Book1_duplicate.xlsx	Byte-for-byte identical copy (same checksum)
configs/model/topic_modeling/, model/classification/	configs/topic_modeling/, configs/classification/	Same task names as scripts/ and src/

Code edits (paths only, plus one bug fix)

File	Edit
02_prepare_rvl_cdip_corpus.py, fit_bertopic.py	ROOT = parents[1] -> parents[2] (files are one folder deeper)
02_prepare_rvl_cdip_corpus.py, test_ocr_one.py	Import changed from src.ocr_client to intelligent_document_finetuning.ocr.client
ocr/client.py	RUNNER path now points to backend_runner.py in the same folder
both notebooks	Excel path changed to ../data/raw/pao_metadata/Book1.xlsx
extract_one_rvl_cdip_image.py	**Bug fix:** saved to data/raw/**rv1**_cdip (number one) instead of rvl_cdip
Usage comments in scripts	Updated the 'Run:' lines to the new paths

Removed

- Empty **__init__.py** files in folders that are not Python packages (configs/, data/, scripts/, src/, test/). __init__.py means 'this folder is importable Python code'; in data/ or configs/ it was misleading. Empty data folders are now kept in git with **.gitkeep** files instead (.gitignore updated to match).
- Empty, unused package folders: export/, models/llm/, training/, utils/, tools/, and data/train_splits/.
- Empty configs/trainer/setfit_trainer.yaml (trainer settings now live in configs/classification/setfit.yaml).
- Stale __pycache__ folders (Python regenerates them automatically).
- test/ renamed to tests/ (the common convention).

Added

- **README.md**: folder map and all run commands with the new paths (your old commands are kept, updated).
- **configs/*.yaml**: filled with the values your code and notes actually use. The scripts don't read these yet; they record settings in one place (configs/README.md explains this).
- **A docstring in each package __init__.py** saying what belongs in that folder.
- **notes/classification_runs.md**: your baseline results copied from the notebook outputs, plus the open issues.
- pyproject.toml: a real project description.

Things to know

Nothing is committed. Review the changes in GitHub Desktop or VS Code, then commit from Windows. If you dislike anything, discard the changes there and everything returns to how it was (all moved files were tracked by git). Two leftovers for you to decide on: **_to_delete/Book1_duplicate.xlsx** (safe to delete) and an empty **state/** folder created on 20 Oct that I left alone because I didn't create it.

6. How to run things now

Always run from the repo root folder.

```
# RVL-CDIP pipeline
python scripts/rvl_cdip/01_download_rvl_cdip.py
python scripts/rvl_cdip/02_prepare_rvl_cdip_corpus.py --per-class 100

# BERTopic experiments
python scripts/topic_modeling/fit_bertopic.py --run-name v2_mcs10 --min-cluster-size 10

# OCR smoke test (needs the Intelligent-Document-Search venv)
python scripts/debug/test_ocr_one.py

# PAO notebooks: open Jupyter, then run them from the notebooks/ folder
```

7. Where do I put new things?

I want to...	Put it in
Try an idea or look at data	notebooks/03_....ipynb (next number)
Reuse a function in more than one place	src/intelligent_document_finetuning/<task>/
Run a full experiment from the terminal	scripts/<task>/
Record a setting (learning rate, threshold, seed)	configs/<task>/....yaml
Save a cleaned dataset	data/processed/ (e.g. pao_metadata_v1.parquet)
Save a trained model	models/<model_type>/<run_name>/
Write down what a result means	notes/<experiment>_runs.md
Test something quickly, then throw it away	scripts/debug/

8. Suggested next steps for the structure

- Move the PAO cleaning steps from notebook 01 into **src/.../data/pao_metadata.py** with a function like **load_pao_metadata()** that returns (text, label), and save its output to data/processed/.
- Put the TF-IDF + LR pipeline and SetFit into **src/.../classification/**, both with the same **fit()** and **predict_proba()** methods. Same interface means the agent can call either one as a tool, and your comparison is fair.
- Put StratifiedGroupKFold + macro-F1 into **src/.../evaluation/** so every model is scored the same way.
- Later, make scripts read their settings from configs/ instead of hard-coded values or long command lines.
- When adding SetFit, check it installs on your Python 3.14 / transformers 5.x setup first.